

Franklin Templeton AI/ML Engineer — Complete Preparation Brief

Role: Digital & AI/ML Engineer (Franklin Templeton) **Approach:** Project-driven, FT-specific, depth-first

0. Read this first

The standard advice for AI/ML interviews is generic. You are interviewing at a \$1.66 trillion AUM asset manager that has already moved past the experimentation phase. Their AI team has shipped real systems, has named partnerships, and uses a specific stack. If you walk in talking about "RAG" generically, you sound like every other candidate. If you walk in able to discuss why they chose LCEL + contextual compression retrievers over naive RAG, you become memorable.

This brief is structured so that by the time you finish the projects, you will have built a portfolio that mirrors what Franklin Templeton's AI Platform team actually builds in production. You will also have absorbed the theory through doing, which is the only way it sticks.

One honest note: your OnEasy.ai work (multi-agent CA firm with governance layers, audit trails, provenance tracking) is almost a 1:1 match for what FT is doing with Wand AI. You should treat that as your single strongest narrative asset. We will use it.

1. Franklin Templeton AI intelligence brief

This is not from their JD. This is reconstructed from press releases, partnerships, executive statements, and public LinkedIn signals. Use it to ground every interview answer.

1.1 The people who matter

Deep Srivastav — Chief AI Officer

- Long-time AI leader at Franklin Templeton, co-author on academic work in AI-driven wealth management personalization, Markowitz Award recipient.
- Public philosophy: warns against "AI tourism" (isolated experiments that produce demos without operational impact). Frames AI work as "re-engineer workflows so humans and algorithms continuously learn from each other."

- Interview signal: if you sound like a demo-builder, you fail. If you sound like someone who thinks in workflows + measurable business outcomes, you stand out.

Vasundhara Chetluru — Head of AI Platform

- Background: TIFIN, Magnifi, Jump Trading, Teza Technologies, Manifold Partners, Fermilab, Brookhaven. Deep quant + platform background.
- Public technical signals (from her LinkedIn posts): builds with LCEL-based RAG architectures using LangChain, uses contextual compression retrievers, evaluates with LLM-powered metrics (faithfulness, relevance, correctness), thinks about Cache-Augmented Generation (CAG) vs RAG tradeoffs, modular embedding/LLM selection across OpenAI, HuggingFace, Mistral.
- Direct quote you can paraphrase: "AI is not a single capability. Organizations must create an intelligent system using their own data that integrates with existing workflows and represents their organizational value proposition."
- Interview signal: this person values architectural taste. Know your RAG variants, know when you would not use RAG, and know evaluation.

1.2 The partnerships (what they actually run on)

Partner	What it gives FT	Why it matters for your interview
Wand AI	Agentic workforce, Autonomous Workforce and Agent Management. Agents register as workforce members, go through training, continuously improve. Production across multiple divisions, scaling broadly in 2026.	This is exactly the OnEasy.ai architecture. You can speak to agent governance, orchestration, continuous learning loops.
Microsoft	Financial AI platform built on composable business applications, reasoning capabilities over structured financial data, personalized client experiences.	Azure OpenAI stack. Mention Semantic Kernel or AutoGen knowledge if relevant.
Articul8 on AWS	Full-stack GenAI platform, autonomous and multi-modal	Bedrock, SageMaker, multi-modal pipelines. AWS is the cloud they live on.

Partner	What it gives FT	Why it matters for your interview
	applications across the asset management value chain.	

1.3 The team you would likely join

Pulled from the Fiduciary Trust International (Franklin Templeton company) job posting: **"The AI, Data, and Process Optimization team builds solutions that combine data engineering, analytics, and generative AI to deliver measurable business impact."**

Read that sentence twice. The order is intentional:

1. **Data engineering** comes first. They believe AI without clean data is theater.
2. **Analytics** comes second. Classical ML and statistics matter.
3. **Generative AI** comes third. It is a layer, not the foundation.
4. **Measurable business impact** closes the sentence. Every system must justify itself in ROI terms.

If you talk like a GenAI hammer looking for nails, you fail. If you talk like a workflow engineer who happens to use AI, you fit.

1.4 The active problem space at FT (what to build for)

Based on Deep Srivastav and Vasundhara's public commentary plus FT press releases, these are the live use cases:

1. **Investment research augmentation** — analyst copilots over earnings transcripts, broker reports, 10-Ks, news.
2. **Regulatory and compliance document QA** — grounded answers over policy documents with citation.
3. **Personalized client experiences** — combining structured financial data with contextual reasoning.
4. **Operations automation** — internal workflows in distribution, sales, ops.
5. **Goals optimization** — older but still live — ML for portfolio goal achievement (the "Goals Optimization Engine").
6. **ESG analysis** — they have public ESG strategies, NLP over sustainability reports is active.

The single highest-leverage project you can build that maps to this: a financial document RAG system with citation, evaluation, and a small agent layer. We will build it.

2. Your narrative (how to position)

Here is the narrative arc you walk into interviews with. Internalize it.

Past: Strong Python and competitive programming foundation (ICPC AIR 14 if it comes up — do not lead with it but do not hide it). Founding engineer experience shipping production systems. Built and operated multi-agent AI systems in a regulated domain (finance/tax).

Present: Building OnEasy.ai, a multi-agent AI-operated CA firm. The system handles live tax filings, entity incorporations, financial modeling. The architecture uses multi-agent orchestration with governance layers, state persistence, audit trail emphasis, and provenance tracking. This is the same problem class Franklin Templeton is solving with Wand AI for investment research and operations.

Why Franklin Templeton: You want to work on AI in a regulated environment at scale. OnEasy works with Indian tax compliance (one regulator, mid-volume). Franklin Templeton operates across 150+ countries, \$1.66T AUM, multiple regulators, with the same fundamental engineering challenge (agents that act on financial data with auditable correctness). The scale, governance discipline, and depth of the AI Platform team is why.

What you bring: Practical experience designing agentic systems that have to be right, not just plausible. Familiarity with the exact architectural patterns (orchestration, state, governance, eval) the FT AI Platform team uses.

What you want to learn: Classical ML depth, financial domain modeling, production MLOps at FT's scale, exposure to fixed income, equity research, and multi-asset workflows.

That is the entire pitch. Three minutes. Practice it.

3. Interview process at Franklin Templeton (what to expect)

Reconstructed from Glassdoor, Indeed, Interview Query, Wall Street Oasis data on FT.

Process length: ~1 month, 4 to 5 rounds typical.

Round 1: HR Screen (30–45 min)

- Walkthrough of resume.
- "Why Franklin Templeton?" (have a real answer — see Section 2).

- "Why this role?"
- Behavioral basics. STAR method.

Round 2: Hiring Manager / Tech Round 1 (45–60 min)

- Project deep-dive on one of your resume projects. Expect 20–30 minutes on a single project.
- Foundational ML/NLP questions.
- "How would you approach X problem here?"

Round 3: Tech Round 2 (45–60 min)

- Hands-on coding or system design.
- Python coding (Pandas, basic algorithms, API design).
- AI system design (build RAG, scale to N documents, evaluate hallucinations).

Round 4: Bar Raiser / Senior Panel (45–60 min)

- Cross-functional. Could include UX, solutions architect, or PM.
- Behavioral plus high-level technical.
- Communication and culture-fit weighted heavily.

Round 5 (sometimes): Final with senior leader

- For some teams, final round with CTO or AI Platform leadership.

Domain questions you must be able to answer (these are FT-specific, asked frequently):

- What is an ETF? Difference vs mutual fund?
- Active vs passive management?
- What is alpha? Beta?
- Why asset management over banking, hedge funds, or pure tech?

You do not need finance depth. You need conversational literacy. Spend 4 hours on these basics, no more.

4. The 12 Projects

Each project is structured as:

1. **What you build** (one paragraph)

2. **Why FT cares** (the business mapping)
3. **Prerequisites** (what to learn on YouTube/docs before starting)
4. **Architecture and stack**
5. **Depth — what you must understand at code/theory level**
6. **Stretch additions** (to make it stand out)
7. **Interview hooks** (what to say about it)

Projects are ordered by leverage. Build Project 1 fully before moving on. If you only build the first 5, you are already interview-ready for FT.

PROJECT 1: Financial Document Intelligence RAG (the flagship)

What you build: An end-to-end system that ingests 10-K filings, earnings call transcripts, and broker research reports (all freely available from SEC EDGAR and company IR sites). User asks questions in natural language. System returns grounded answers with citations, page numbers, and confidence scores. Built with a real evaluation harness.

Why FT cares: This is the most common live use case at every asset manager including Franklin Templeton. Vasundhara Chetluru's own public RAG demo is built on regulatory document QA. If you walk in with this project, you are walking into her wheelhouse.

Prerequisites (study before building, 3 to 5 days):

- Python intermediate: classes, decorators, async basics, type hints
- HTTP and REST API fundamentals: GET/POST, status codes, JSON, headers
- FastAPI basics: routing, Pydantic models, dependency injection
- Vector spaces and cosine similarity (intuition, not full linear algebra)
- Tokenization (BPE, WordPiece) — what they do, why they matter
- Embedding intuition — sentence-transformers, OpenAI ada/text-embedding-3
- LangChain core abstractions: Document, Retriever, Chain, LCEL
- One vector DB end-to-end: Chroma (easiest) or FAISS (no server)
- Prompt engineering basics

Architecture and stack:

[PDF/HTML ingest] -> [Chunker with semantic splitting] -> [Embeddings] -> [Vector DB]

|

[User Query] -> [Query rewrite/HyDE] -> [Hybrid retrieval: BM25 + dense] -> [Reranker]

|

[Contextual compression] -> [LLM with citation prompt]

|

[Evaluation layer]

Stack:

- **Backend:** FastAPI + Pydantic
- **Orchestration:** LangChain with LCEL (this is what FT uses, learn it specifically)
- **Vector DB:** ChromaDB locally, then Pinecone or Weaviate for the cloud version
- **Embeddings:** `text-embedding-3-small` for cost, `bge-large-en-v1.5` for an open-source comparison
- **LLM:** OpenAI GPT-4o for primary, Llama 3.1 8B via Together AI or local Ollama for cost-aware fallback
- **Reranker:** `cross-encoder/ms-marco-MiniLM-L-12-v2` or Cohere Rerank
- **Eval:** RAGAS framework (faithfulness, answer relevance, context precision, context recall)
- **UI:** Streamlit for fast iteration, then a small React frontend
- **Document parsing:** `unstructured` library for PDFs, `pdfplumber` for tables

Depth — what you must understand:

Chunking strategy: Why fixed-size chunking destroys financial documents (tables, footnotes, MD&A sections). Implement semantic chunking with `unstructured`'s layout-aware parser. Be able to explain when you would use 256-token chunks vs 1024-token chunks vs hierarchical chunking.

Hybrid retrieval: Pure vector search misses exact numbers and ticker symbols. Implement BM25 sparse retrieval alongside dense embeddings, then fuse with reciprocal rank fusion (RRF). Be able to explain RRF math.

Contextual compression: Vasundhara's exact phrase. After retrieving top-k chunks, use an LLM call or a smaller embedding-based filter to remove irrelevant sentences before passing to the generator. Reduces hallucination and token cost. Implement with `LLMChainExtractor` or write your own.

Citation generation: Force the model to cite chunk IDs in its output. Parse the output and surface the actual source spans. This is non-negotiable for finance.

Evaluation:

- **Faithfulness:** Does the answer follow from the retrieved context? (LLM-as-judge.)
- **Answer relevance:** Does the answer address the question?
- **Context precision:** Are the retrieved chunks the right ones?
- **Context recall:** Did we retrieve everything needed?
- Build a small eval set of 30 to 50 question-answer pairs over a real 10-K. Score before and after every change.

Latency and cost: Track p50/p95 latency, tokens in/out per query, dollar cost per query. Finance cares about cost-per-query at scale.

Stretch additions:

- Cache-Augmented Generation (CAG) mode: for a single 10-K small enough to fit in context, skip retrieval and load the whole doc into the context window. Compare latency, cost, and accuracy vs RAG. Vasundhara has posted about CAG vs RAG specifically — having a working comparison is a direct hook.
- Streaming responses with server-sent events.
- Multi-document retrieval with metadata filtering (filter by ticker, date range, document type).
- Table-aware retrieval: extract tables separately, index them with structured retrieval.

Interview hooks:

- "I built this because FT's regulatory and earnings document workflows are the most leveraged GenAI use case in asset management."
- "I implemented hybrid retrieval because pure dense embeddings miss numerical lookups and ticker references."
- "On a 50-question eval set, I got faithfulness from 0.71 to 0.89 after adding contextual compression and a reranker. Here is the breakdown."
- "I built a CAG comparison after reading Vasundhara Chetluru's post on it. For single-document QA, CAG won on latency. For multi-document or large corpora, RAG won on accuracy and cost."

PROJECT 2: Earnings Call Sentiment and Surprise Detector

What you build: Ingest earnings call transcripts (Seeking Alpha, Motley Fool transcripts, or scraped from company IR pages). Classify management tone, detect hedging language, score forward-looking statements vs historical statements, and detect "surprise" — when language sentiment diverges from reported financial results.

Why FT cares: Equity research analysts at FT spend hours reading transcripts. NLP-driven sentiment with financial-domain models is a known alpha source. This shows you understand classical NLP, finance-tuned models, and you can think about signal generation.

Prerequisites (2 to 3 days):

- Hugging Face Transformers: pipelines, tokenizers, models
- FinBERT model family (yiyanghkust/finbert-tone, ProsusAI/finbert)
- Pandas advanced: groupby, time series, merge
- Basic time series alignment (price reaction windows around earnings dates)
- Simple statistical hypothesis testing

Architecture and stack:

- **Data:** scrape or download 200+ earnings transcripts across 20 companies, 4 quarters each
- **Models:** FinBERT for sentiment, a custom rule-based hedging detector (use spaCy for matchers), an LLM call for forward-looking vs historical classification
- **Storage:** PostgreSQL with a clean schema (companies, transcripts, sentences, scores)
- **Analytics:** event-study around earnings date — measure abnormal return in (-1, +1) day window vs sentiment score

Depth:

- Why FinBERT and not vanilla BERT (financial domain pretraining matters)
- Sentence-level vs section-level sentiment (MD&A vs Q&A section)
- Hedging language taxonomy: epistemic, deontic, dynamic modality
- Event study methodology: cumulative abnormal return (CAR), market model regression
- The relationship between text-based signals and price movement (do not overclaim causality)

Stretch:

- Compare FinBERT vs prompting GPT-4 with a few-shot sentiment prompt. Plot accuracy/cost tradeoff.
- Build a small backtest: long top-quintile-sentiment companies, short bottom, see if there's any signal. Even if it doesn't work, the analysis is impressive.

Interview hooks:

- "I built this to understand whether NLP-extracted signals from earnings calls correlate with stock returns. On my sample, there was a measurable but noisy 0.X correlation."

- "Choosing FinBERT over general BERT mattered because financial language uses domain-specific tone (guidance, headwinds, softness)."
-

PROJECT 3: Multi-Agent Investment Research Assistant

What you build: A multi-agent system where specialized agents collaborate on an investment thesis. Agents include: Data Gatherer (pulls financials, news, transcripts), Fundamental Analyst (computes ratios, peer comparison), Sentiment Analyst (uses Project 2), Risk Analyst (extracts risk factors from 10-K), and Report Writer (synthesizes). An Orchestrator coordinates them with state persistence and an audit log.

Why FT cares: This is literally what Wand AI does for them. If you can build a working version, you can speak intelligently about their production architecture. This is also the project where your OnEasy.ai background pays off most.

Prerequisites (4 to 5 days):

- LangGraph (the StateGraph abstraction)
- Function calling / tool use with OpenAI or Anthropic APIs
- AsyncIO basics in Python
- State management patterns (event sourcing intuition)
- Pydantic for typed agent messages

Architecture and stack:

- **Framework:** LangGraph for the orchestration graph (preferable to CrewAI for production patterns; FT-level engineering favors graph-based control flow)
- **State:** typed Pydantic state object, persisted to SQLite or Redis between steps
- **Agents:** each is a function node with a system prompt, a set of tools, and an output schema
- **Tools:** SEC EDGAR API, yfinance for prices, NewsAPI, your Project 1 RAG endpoint as a tool
- **Observability:** LangSmith for tracing, or your own structured logging with run IDs

Depth:

- Why graph-based orchestration beats role-playing crews for production (control, debuggability, retries)
- State persistence and resumability (an agent run that takes 90 seconds can fail at second 80; you need to resume, not restart)
- Tool selection patterns: when does the LLM choose vs when do you hard-route?

- Failure modes: hallucinated tool calls, infinite loops, context overflow
- Governance: every agent decision should be logged with input, output, tool calls, and reasoning trace. This is exactly the "audit trail emphasis" from OnEasy.

Stretch:

- Add a Critic agent that reviews the final report and sends it back if quality is low.
- Add cost tracking per run.
- Add human-in-the-loop checkpoints for high-impact decisions.

Interview hooks:

- "I built this because the FT-Wand AI partnership is doing exactly this for investment research, and I wanted to understand the architectural tradeoffs firsthand."
- "Graph-based orchestration over role-playing because production needs determinism and observability, not emergent behavior."
- "Every agent action is logged with a run ID, input state, tool calls, and output. That is the same governance pattern I built for OnEasy.ai's tax workflows."

PROJECT 4: Portfolio Construction with Classical ML

What you build: Using historical price and fundamental data on S&P 500 stocks, build a model that predicts cross-sectional returns. Use this to construct long-short portfolios. Compare a classical approach (gradient-boosted trees with engineered features) against a simple deep learning approach.

Why FT cares: Demonstrates you can do classical ML, not just GenAI. FT has quantitative strategies. This signals you understand the supervised learning lifecycle.

Prerequisites (4 to 5 days):

- scikit-learn fully: pipelines, ColumnTransformer, cross-validation
- XGBoost or LightGBM
- Feature engineering for time series (lags, rolling means, momentum, value, quality factors)
- Walk-forward validation (not random k-fold — important)
- Sharpe ratio, max drawdown, information ratio

Architecture:

- **Data:** monthly returns + fundamentals (P/E, ROE, debt/equity, growth) for 500 stocks over 10 years
- **Features:** Fama-French style factors + your own
- **Models:** XGBoost regressor, then a small MLP, then a Transformer-for-tabular (TabTransformer) as a stretch
- **Validation:** expanding window walk-forward — train on years 1 to N, test on year N+1, advance, repeat
- **Backtest:** monthly rebalanced long top decile, short bottom decile, compute Sharpe ratio

Depth:

- Why walk-forward and not random k-fold (look-ahead bias is the silent killer in financial ML)
- Bias-variance tradeoff in financial ML context (signal-to-noise is brutally low)
- Why most "alpha" published in papers fails in production (overfitting, transaction costs, regime change)
- Regularization: L1 vs L2 in financial features (L1 helps with redundant factor exposures)
- SHAP values for feature attribution

Stretch:

- Add transaction cost modeling.
- Add a regime detection layer (HMM or k-means on macro indicators) and condition the model on regime.

Interview hooks:

- "I wanted to understand classical ML in the FT context. I learned that the engineering challenge is not the model, it is preventing look-ahead bias and handling regime change."
- "Walk-forward validation is non-negotiable in financial ML. Random k-fold leaks the future into training."

PROJECT 5: ESG Document Analyzer

What you build: Ingest sustainability reports (CDP, CSR reports, SASB disclosures). Extract structured ESG metrics, classify against SASB or GRI taxonomies, detect greenwashing patterns (language vs actual disclosures), and generate a structured score.

Why FT cares: Franklin Templeton has multiple ESG investment strategies. NLP over sustainability reports is an active workstream. Niche enough that few candidates build it. Memorable.

Prerequisites (2 to 3 days):

- spaCy for entity recognition and rule-based matching
- LLM structured output (function calling / JSON mode)
- Basic information extraction patterns

Architecture:

- **Data:** 30 CSR/sustainability reports from S&P 100 companies
- **Pipeline:** parse PDF → segment into ESG topics → extract numerical disclosures with regex+LLM → classify against SASB taxonomy → score
- **Output:** structured JSON per company with comparable metrics

Depth:

- Information extraction vs generation (when to use regex, when LLM, when both)
- SASB vs GRI vs TCFD frameworks (just enough literacy to discuss)
- Greenwashing detection patterns: high commitment language with low specific disclosure

Stretch:

- Cross-company comparison dashboard.
- Time-series of ESG disclosures for one company over 5 years.

Interview hooks:

- "I built this because FT has ESG-oriented strategies, and the cost of analyst time on sustainability reports is high."

PROJECT 6: Production-Grade FastAPI + Docker Deployment

What you build: Take Project 1 (the RAG system) and ship it as a production service. Containerized, with authentication, rate limiting, logging, health checks, and deployed to AWS (EC2 + ECR, or App Runner for simplicity).

Why FT cares: This is your "I can deploy" credential. AI engineers who cannot ship are not useful at scale.

Prerequisites (3 to 4 days):

- Docker basics: Dockerfile, multi-stage builds, docker-compose
- Linux basics: bash, systemd, journalctl
- AWS basics: IAM, EC2, S3, ECR, CloudWatch
- nginx as a reverse proxy
- JWT-based auth in FastAPI
- Environment management (12-factor app principles)

Architecture:

- FastAPI app with `/query`, `/upload`, `/health`, `/metrics` endpoints
- JWT auth middleware
- Rate limiting with `slowapi`
- Structured JSON logging
- Multi-stage Dockerfile (builder image + slim runtime)
- Docker Compose for local: app + ChromaDB + nginx
- Deployed to AWS App Runner or EC2 with a CI/CD pipeline via GitHub Actions

Depth:

- Why multi-stage Docker builds (image size, security)
- Async vs sync FastAPI endpoints (when each is appropriate for LLM workloads)
- Streaming responses (SSE for token-by-token generation)
- Idempotency and retries
- Health checks: liveness vs readiness
- Observability: structured logs, request IDs, latency histograms

Stretch:

- Add Prometheus metrics export + a Grafana dashboard.
- Add a queue (Redis or SQS) for long-running ingestion jobs.

Interview hooks:

- "Here is the live URL. Hit `/query` with auth token X."
 - "I went with App Runner over EC2 for the deploy because rollback and auto-scaling were free, and for a small service the cost is similar."
-

PROJECT 7: Hallucination Detection and Eval Harness

What you build: A standalone evaluation framework that tests any RAG system against a question-answer dataset and reports faithfulness, answer relevance, context precision, context recall, and hallucination rate. Compare LLM-as-judge against simpler heuristics.

Why FT cares: Evaluation is the #1 missing skill in most candidates. In regulated finance, "the model works" is not enough — you must prove it works and quantify when it fails.

Prerequisites (2 days):

- RAGAS framework
- DeepEval framework
- Statistical concepts: precision, recall, F1, calibration

Architecture:

- Test set: 100 manually written Q&A pairs over 5 financial documents
- Eval runners for RAGAS, DeepEval, and your own
- Comparison report

Depth:

- LLM-as-judge: prompt design, judge bias, position bias, self-preference
- When LLM-as-judge fails (subjective questions, long contexts)
- Heuristic alternatives: BLEU, ROUGE, NLI-based entailment (Hugging Face HHEM model)
- Hallucination taxonomy: intrinsic vs extrinsic, faithfulness vs factuality

Interview hooks:

- "I built this because I noticed RAG benchmarks differ wildly across frameworks. My harness lets me compare metrics on the same test set."
- "On my financial QA test set, RAGAS-faithfulness and HHEM agreed 78% of the time. The disagreements were mostly edge cases involving aggregated numerical answers."

PROJECT 8: SQL Copilot for Financial Database

What you build: A text-to-SQL system over a sample financial database (you can use a free dataset like FINRA TRACE bonds or scaffold one from yfinance). User asks "what was the

top-performing tech stock in 2023?" — system writes and executes SQL, returns answer with the query shown for auditability.

Why FT cares: Internal analyst tooling is a huge area. SQL copilots are deployed at every major asset manager.

Prerequisites (2 to 3 days):

- SQL intermediate (joins, window functions, CTEs)
- LLM structured output
- SQLAlchemy
- Schema introspection

Architecture:

- SQLite or Postgres with a real financial schema
- LLM call with schema-aware prompt (include only relevant tables based on a semantic retriever)
- Query validation (parse with sqlparse, reject DDL/DML, check column existence)
- Execute and format

Depth:

- Why schema retrieval beats stuffing the whole schema in the prompt (token cost, accuracy)
- SQL injection in LLM context (your query validator is the only defense)
- Few-shot example selection: which examples to include is itself a retrieval problem

Stretch:

- Add query repair: if the SQL fails, feed the error back to the LLM and let it fix.
- Add result visualization with a chart-type-selection LLM call.

PROJECT 9: Fine-Tune a Small Domain Model with LoRA

What you build: Take a small open model (Llama 3.1 8B or Mistral 7B), and fine-tune it with LoRA on a financial Q&A dataset. Compare against the base model and against GPT-4 prompted.

Why FT cares: Fine-tuning is one of the questions on Vasundhara's mental model (when to RAG, when to fine-tune, when to CAG). Having done it once is enough.

Prerequisites (3 to 5 days):

- Hugging Face Transformers + PEFT + TRL
- LoRA math (rank, alpha, target modules — intuition not derivation)
- QLoRA (4-bit quantization)
- Google Colab Pro or a cloud GPU (an A100 or even a T4 is enough for QLoRA)
- Dataset preparation (instruction tuning format)

Architecture:

- Dataset: 500 to 1000 Q&A pairs from public finance sources (FinGPT datasets, or generate synthetically with GPT-4 then human-filter)
- Train with TRL's SFTTrainer
- Eval on a held-out 100 examples

Depth:

- LoRA mechanism: why low-rank adapters work
- When fine-tuning beats prompting (consistent format, domain language, latency, cost at high volume)
- When fine-tuning loses to RAG (knowledge updates, factual queries)
- Catastrophic forgetting risk

Interview hooks:

- "I fine-tuned with QLoRA on a single T4 GPU. Final model improved exact-match on financial format questions from 41% to 67% vs base. Still lost to GPT-4 by 8 points but was 30x cheaper per query."

PROJECT 10: Real-Time News + Filing Alert System

What you build: Stream news headlines and SEC filing announcements, classify them by company and event type (M&A, earnings, regulatory, leadership change), and route alerts based on a portfolio holdings list. Use embeddings to deduplicate similar headlines.

Why FT cares: Internal portfolio manager tooling. Latency-sensitive AI deployment. Shows you can think about streaming systems.

Prerequisites (3 days):

- Async Python (asyncio, aiohttp)
- A streaming pattern (Kafka is overkill; use Redis Streams or simple WebSocket polling)
- Lightweight classification (could be embeddings + cosine, or a small fine-tuned classifier)

Architecture:

- Sources: NewsAPI free tier, SEC EDGAR RSS, an Alpaca markets news endpoint
- Pipeline: poll → embed → cluster (dedupe) → classify event type → match to portfolio → emit alert
- Storage: Redis for hot state, Postgres for history
- Delivery: webhook + email

Depth:

- Streaming vs batch tradeoffs
- Deduplication via embedding similarity threshold
- Cold-start problem for new tickers
- Latency budget: under 30 seconds end to end from publication to alert

PROJECT 11: MLOps Pipeline for the Portfolio Model (Project 4)

What you build: Take Project 4 (the classical ML portfolio model) and wrap it in a full MLOps pipeline: data versioning, experiment tracking, model registry, deployment, monitoring for drift.

Why FT cares: Production ML lifecycle is what they actually care about for the ML side of the role.

Prerequisites (4 days):

- MLflow (tracking, registry, serving)
- DVC for data versioning
- Evidently AI or NannyML for drift detection
- GitHub Actions for CI/CD

Architecture:

- DVC tracks the dataset versions
- MLflow logs every training run with hyperparams, metrics, artifacts
- Model registry: dev → staging → prod promotion gates
- Deployment: FastAPI endpoint that loads the prod model from MLflow

- Monitoring: Evidently runs daily, compares prod input distribution vs training distribution, alerts on drift

Depth:

- Data drift vs concept drift (and which one matters more in finance)
 - Why model versioning needs to be coupled with data versioning
 - Reproducibility: same data + same code + same env should yield bit-identical models (or close)
 - Online vs offline evaluation
-

PROJECT 12: Long-Context Document Comparator

What you build: A system that compares two versions of a long document (e.g., this year's 10-K vs last year's) and surfaces material changes — what was added, removed, or modified. Beyond simple diff: detect *semantic* changes, classify importance, and summarize.

Why FT cares: Analysts read 10-Ks year over year specifically to find what changed. This is a real and unsexy problem that consistently delivers value.

Prerequisites (2 to 3 days):

- Long-context LLMs (Claude 3.5 Sonnet, GPT-4 Turbo, Gemini 1.5)
- Document alignment algorithms (sequence alignment intuition)
- Diff algorithms (Myers diff at a high level)

Architecture:

- Section-aware document parsing (split by 10-K item numbers)
- Per-section diff: textual diff + semantic comparison via LLM
- Classification: cosmetic vs material change
- Output: a structured report

Depth:

- Token cost management for long-context (chunk + reassemble vs single-shot)
 - When semantic diff matters vs when textual diff is enough
-

5. Theory you must hold in working memory

The projects teach this, but you should be able to *talk* about all of it cleanly. One-line summaries per concept follow.

Machine learning fundamentals

- **Supervised vs unsupervised vs reinforcement:** labels present, labels absent, reward signal.
- **Bias-variance tradeoff:** underfitting vs overfitting; total error = bias² + variance + irreducible.
- **Overfitting countermeasures:** more data, regularization (L1/L2), early stopping, dropout, cross-validation.
- **Cross-validation:** k-fold for IID data, walk-forward for time series. **Never random k-fold on financial data.**
- **Precision/Recall/F1:** $P = TP/(TP+FP)$, $R = TP/(TP+FN)$, F1 is harmonic mean. Use precision when false positives are costly; recall when false negatives are.
- **ROC-AUC vs PR-AUC:** ROC-AUC for balanced, PR-AUC for imbalanced.
- **Imbalanced datasets:** SMOTE, class weights, focal loss, threshold tuning.

Deep learning

- **Backpropagation:** chain rule applied to compute gradients of loss w.r.t. weights.
- **Activation functions:** ReLU (default), GELU (transformers), softmax (classification head).
- **Loss functions:** MSE (regression), cross-entropy (classification), contrastive (embeddings).
- **Optimizers:** SGD with momentum, Adam (adaptive learning rate per param). AdamW for transformers.

NLP and LLMs

- **Tokenization:** BPE (GPT), WordPiece (BERT), SentencePiece (Llama). Subword units, not words.
- **Embeddings:** dense vectors capturing semantic meaning. Word2Vec → contextualized (BERT) → sentence (sentence-BERT).
- **Self-attention:** every token attends to every other token, weighted by learned similarity. Output is a context-aware representation.
- **Transformer:** stacked attention blocks + feedforward + residual connections. Encoder-only (BERT), decoder-only (GPT), encoder-decoder (T5).
- **BERT vs GPT:** BERT is bidirectional, trained on masked language modeling, good for understanding. GPT is autoregressive, trained on next-token prediction, good for generation.

- **Context window:** how many tokens the model can attend to. GPT-4 Turbo 128k, Claude Sonnet 200k, Gemini 1.5 Pro 2M.
- **Temperature, top-k, top-p:** sampling controls. Temperature scales logits before softmax. Top-k truncates to top-k tokens. Top-p (nucleus) truncates to smallest set with cumulative prob $\geq p$.

RAG-specific

- **Why RAG:** ground LLMs in retrievable knowledge, avoid retraining for new info, enable citations.
- **Chunking:** splitting documents for retrieval. Fixed-size is naive, semantic chunking respects structure.
- **Dense vs sparse retrieval:** dense (embeddings + cosine) captures semantics, sparse (BM25) captures exact matches. Hybrid is usually best.
- **Reranking:** retrieve k=50, rerank to k=5 with a cross-encoder for precision.
- **Contextual compression:** filter retrieved chunks to only sentences relevant to the query before generation.
- **HyDE (Hypothetical Document Embeddings):** generate a hypothetical answer, embed it, retrieve with that. Often improves recall.
- **Query rewriting:** rephrase the user query for retrieval (decomposition, expansion).

Evaluation

- **Faithfulness:** answer follows from context.
- **Answer relevance:** answer addresses the question.
- **Context precision/recall:** retrieval quality.
- **LLM-as-judge biases:** position bias, self-preference, verbosity bias.

Fine-tuning

- **Full fine-tuning:** update all weights. Expensive.
- **LoRA:** add low-rank matrices, freeze original weights. ~1% of params trained.
- **QLoRA:** LoRA + 4-bit quantized base weights. Fits 70B on a single 48GB GPU.
- **When to fine-tune vs RAG:** fine-tune for format/tone/domain language; RAG for knowledge.

Production ML

- **Drift:** data drift (input distribution changes), concept drift (relationship between input and output changes).
 - **Model serving:** batch (precompute) vs online (real-time).
 - **A/B testing:** how you validate that the new model is actually better in prod.
-

6. Interview question bank (with FT framing)

Foundational ML

1. Explain bias-variance tradeoff. *Have a financial example: a very simple factor model has high bias but is stable; a deep model on the same data has low bias but high variance and overfits.*
2. Walk me through how you'd diagnose overfitting in a model. *Train-val gap, learning curves, regularization paths.*
3. What's the difference between L1 and L2 regularization? *L1 produces sparse weights, useful for feature selection in finance. L2 shrinks uniformly.*
4. Why might you choose XGBoost over a neural network? *Tabular data, interpretability, smaller datasets, robust to feature scaling.*
5. Explain ROC-AUC. *Probability that a random positive is ranked higher than a random negative.*

NLP and transformers

6. Explain self-attention in your own words. *Each token computes a query, key, and value vector. It attends to other tokens by dot-producting its query with their keys, normalizing, and aggregating their values.*
7. Why does positional encoding exist? *Self-attention is permutation-invariant; without position info, "dog bites man" and "man bites dog" are identical.*
8. What's tokenization and why does BPE matter? *Splitting text into subword units. BPE balances vocabulary size and ability to represent unseen words.*
9. BERT vs GPT, when do you use each?
10. What's the difference between fine-tuning and RAG?

RAG and LLM systems

11. Walk me through your RAG architecture. *Use Project 1 here in detail.*
12. How do you evaluate hallucination? *Faithfulness via LLM-as-judge, HHEM, or NLI entailment.*
13. When would you not use RAG? *Single-doc QA fitting in context (CAG wins), highly structured queries (text-to-SQL beats RAG), high-frequency updates with high read volume (cache the answers).*
14. How would you reduce inference cost for a high-traffic RAG system? *Smaller embedding model, fewer retrieved chunks, smaller generator with prompt caching, response caching, request batching, query similarity caching.*
15. What's the failure mode of dense retrieval? *Exact numerical or symbolic lookups (a CUSIP, a specific dollar amount). Hybrid with BM25 fixes this.*

System design

16. Design an enterprise document QA system that scales to 1M PDFs across multiple business units with strict access control.

- Ingestion: distributed pipeline with object storage (S3) as the source of truth.
- Parsing: per-doc-type parsers, queue-based async processing.
- Embedding: batched embedding service, GPU pool with autoscaling.
- Vector DB: sharded by business unit + tenant for access control (or use metadata filters with strict ACL enforcement at retrieval time).
- Retrieval: hybrid (BM25 + dense), reranker, contextual compression.
- Generation: LLM gateway with model routing (small model first, escalate to GPT-4 if needed).
- Governance: every query logged with user, sources retrieved, answer, model version.
- Eval: continuous offline eval on a fixed test set after every model/index update.

17. Design an agentic system that does investment research autonomously.

- Borrow heavily from Project 3. Emphasize state persistence, human-in-the-loop checkpoints, audit trails.

18. Your RAG system has 40% hallucination rate. Diagnose and fix.

- Step 1: define what counts as hallucination. Faithfulness or factuality?
- Step 2: instrument. Log every query with retrieved chunks and answer.
- Step 3: bucket failures. Retrieval failures vs generation failures vs prompt failures.
- Step 4: targeted fixes. Better chunking, hybrid retrieval, reranker, contextual compression, better prompt with "answer only from context", structured output forcing citations.
- Step 5: re-evaluate.

Coding interview prep

You have ICPC AIR 14 background. DSA is not actually your weak point even if you feel rusty. Spend 4 hours brushing up:

- Pandas: filtering, groupby with custom aggs, merge, pivot.
- Python: classes, decorators, generators, context managers.
- One LeetCode-style problem per day for 2 weeks (focus on string manipulation and array problems; FT is not Google).
- Be able to write a simple FastAPI endpoint live in 15 minutes.

Behavioral (STAR)

Prepare 5 stories that each cover multiple competencies:

- **Difficult project:** the OnEasy.ai multi-agent orchestration architecture decision.
- **Failure:** a specific time something broke in production and what you learned.
- **Ambiguity:** an early-stage product decision you had to make with incomplete info.
- **Leadership:** mentoring or leading a small team.
- **Stakeholder communication:** explaining technical decisions to non-technical people.

Finance domain literacy (4 hours)

- **Mutual fund:** pooled vehicle, priced once daily at NAV.
- **ETF:** trades like a stock on an exchange, intraday liquidity, often passive index-tracking.
- **Active vs passive:** active picks securities, passive tracks a benchmark. Passive has eaten active for the last 20 years.
- **Alpha:** return above the benchmark, risk-adjusted.
- **Beta:** sensitivity to market moves.
- **AUM:** assets under management. FT is \$1.66T.
- **Buy side vs sell side:** buy-side manages money (FT), sell-side originates products (Goldman, Morgan Stanley).

"Why Franklin Templeton?" answer template

"Three reasons. One, FT is past the experimentation phase — your AI Platform team has named partnerships with Wand AI, Microsoft, and Articul8, and the AI workforce is already in production. That is rare. Two, the engineering challenge of governed AI agents in a regulated multi-trillion AUM environment is exactly the problem class I have been working on at OnEasy.ai, but at a scale and depth I cannot get on my own. Three, Deep Srivastav's framing of 'no AI tourism, only re-engineered workflows with measurable impact' matches how I think about building."

Customize. Do not memorize this verbatim. But the structure (specificity, alignment, mutual fit) is right.

7. The 90-day roadmap, mapped to projects

The standard 90-day plan in your earlier doc is reasonable but generic. Here is a version that maps directly to the projects above.

Phase 1 — Weeks 1 to 3: Foundation + Project 1

- Days 1 to 5: Python intermediate, FastAPI basics, REST patterns.
- Days 6 to 10: NLP foundations (tokenization, embeddings), LangChain core.
- Days 11 to 21: build Project 1 end to end. Eval harness included.
- Output: live Streamlit demo + GitHub repo + README + a Loom walkthrough.

Phase 2 — Weeks 4 to 6: Project 2 + 3 + 6

- Week 4: Project 2 (sentiment + surprise detector).
- Week 5: Project 6 (FastAPI + Docker + AWS deploy of Project 1).
- Week 6: Project 3 (multi-agent research assistant).
- Output: three projects with deploy URLs.

Phase 3 — Weeks 7 to 9: Classical ML + Eval + Specialty

- Week 7: Project 4 (portfolio construction).
- Week 8: Project 7 (eval harness) + Project 5 (ESG analyzer).
- Week 9: Pick one of Projects 8 / 9 / 10 / 11 / 12 based on what feels most resume-impactful.

Phase 4 — Weeks 10 to 13: Interview prep

- Daily: 1 LeetCode-style problem, 1 system design walkthrough, 1 behavioral story practice.
- Mock interviews: 3 per week. Use Pramp, friends, or your network.
- Finance literacy: 4-hour study sprint, then re-skim weekly.
- Resume: rebuild around projects. See Section 8.

If your interview is sooner than 90 days, prioritize ruthlessly: **Project 1 + Project 2 + Project 6 + interview prep**. Skip the rest. With those three projects you can credibly speak to RAG, NLP, deployment, and evaluation — that covers 80% of likely questions.

8. Resume — what it should actually say

Your resume should not say "Python developer learning AI." It should say "AI engineer building production GenAI systems in regulated domains." The shift is in framing, not in inflation.

Section order

1. Header (name, role tagline, contact, GitHub, LinkedIn, deployed project URLs)

2. About / Tagline (one line — "AI engineer building agentic systems for regulated workflows")
3. Experience (founder of OnEasy.ai, founding engineer roles at SuperU.ai, ClickPe)
4. Projects (the FT-targeted ones — see below)
5. Skills (organized by category)
6. Education

Project block template

[Project Name] — [One-line description] [GitHub URL] [Live URL]

- [Architecture detail with specific tech]
- [Quantitative result — e.g. "improved faithfulness from 0.71 to 0.89 on 50-question eval set"]
- [Production detail — e.g. "deployed to AWS App Runner, p95 latency 1.2s, \$0.003/query"]

Recommended order on resume:

1. Financial Document RAG (Project 1)
2. Multi-Agent Investment Research Assistant (Project 3)
3. OnEasy.ai (your founder work — frame it for AI engineering audience)
4. Earnings Sentiment + Surprise Detector (Project 2)
5. One backend/infra-heavy project (Project 6 outcome)

Skills section

Group by category. No bullet soup.

- **Languages:** Python, SQL, JavaScript/TypeScript
- **ML/DL:** PyTorch, scikit-learn, XGBoost, Hugging Face Transformers
- **GenAI/LLM:** LangChain, LangGraph, OpenAI, Anthropic, Mistral, LoRA/QLoRA fine-tuning
- **Vector + Retrieval:** ChromaDB, Pinecone, FAISS, BM25, hybrid retrieval, rerankers
- **Backend:** FastAPI, async Python, REST APIs, JWT auth, WebSockets
- **Data:** Pandas, NumPy, PostgreSQL, Redis
- **DevOps:** Docker, AWS (EC2, S3, ECR, App Runner), GitHub Actions, MLflow
- **Eval:** RAGAS, DeepEval, LLM-as-judge, custom eval harnesses

Headline language to use

- "Built multi-agent orchestration layer with state persistence, audit trails, and governance hooks" (this is OnEasy in FT language)

- "Designed hybrid retrieval combining dense embeddings and BM25 with cross-encoder reranking"
- "Implemented contextual compression to reduce token cost by 60% and improve faithfulness"
- "Deployed FastAPI service with multi-stage Docker build to AWS, p95 latency under 1.5s"

Language to avoid

- "Leveraged AI to revolutionize..."
 - "Cutting-edge GenAI solutions"
 - "Robust, scalable, end-to-end" (this is generic)
 - Anything with em dashes
-

9. The 14 hardest questions you might get (and how to think about them)

Q1: "How would you build a system that lets a portfolio manager ask questions over 10 years of all 10-K filings in their portfolio?"

Think out loud:

- Scale: 100 companies × 10 years = ~1000 docs, each ~100 pages. Manageable.
- Ingestion: one-time + delta on new filings via EDGAR RSS.
- Parsing: SEC filings have known structure (Item 1, Item 7, etc.). Section-aware chunking.
- Storage: metadata-rich vector DB with filters on ticker, year, section.
- Retrieval: hybrid (BM25 catches CUSIPs and dollar amounts, dense catches semantic queries).
- Generation: cite with ticker + year + section + page.
- Evaluation: a held-out set of analyst questions with known answers.
- Governance: every query logged for compliance review.

Q2: "Your RAG system gives a wrong answer to an analyst about a company's revenue. What do you do?"

- Step 1: reproduce the bug.
- Step 2: inspect the retrieved chunks. Were they correct?
- Step 3: if retrieval was wrong — diagnose: was the right chunk indexed? Was it retrieved? Was it filtered out?

- Step 4: if retrieval was right but the answer was wrong — diagnose the generator. Prompt issue, hallucination, or model limitation?
- Step 5: fix, add to eval set, re-test.
- Step 6: communicate to the analyst with the corrected answer and what changed.

Q3: "When would you fine-tune vs use RAG?"

- Fine-tune for: domain language, consistent output format, latency-critical applications, high query volume where smaller cheaper model wins.
- RAG for: knowledge that updates, factual queries, source citation requirements, large knowledge bases.
- Often both: fine-tune the format, RAG the knowledge.

Q4: "How do you prevent hallucinations in a high-stakes financial application?"

Layer multiple defenses:

- Retrieval quality (hybrid, reranker, contextual compression)
- Prompt: "Answer only from the provided context. If the answer is not present, say so."
- Output structure: force citations to chunk IDs
- Post-hoc check: NLI entailment between answer and context
- Confidence calibration: surface uncertainty
- Human-in-the-loop for material answers
- Continuous eval on a faithfulness test set

Q5: "Walk me through how a transformer works."

Three-minute version:

- Input is tokenized into integers.
- Each token is embedded into a vector.
- Positional encoding is added because attention is order-invariant.
- Each layer has multi-head self-attention: each token attends to every other token, weighted by learned query-key similarity, aggregating values.
- After attention, a feedforward MLP processes each token independently.
- Residual connections + layer norm around each sublayer.
- Stack N layers.
- For generation: the last token's output goes through a softmax over the vocabulary to predict the next token.
- During training: predict every next token, compute cross-entropy loss, backpropagate.

Q6: "Design an evaluation strategy for an AI assistant that helps analysts summarize quarterly results."

- Offline eval: held-out test set of 100 quarterly results with human-written reference summaries. Metrics: ROUGE for surface overlap, LLM-as-judge for quality (factuality, completeness, conciseness), human eval on a 20-sample subset.
- Online eval: track engagement (does the analyst edit the summary?), time-to-completion vs no-AI baseline, downstream decision quality.
- Drift monitoring: weekly re-eval on the held-out set after any model or pipeline change.
- Red-teaming: deliberate adversarial inputs (companies with unusual results, restated financials).

Q7: "Tell me about a time you debugged something hard."

Have a 90-second story. Specific. With a metric.

Q8: "How do you decide what to build vs what to buy?"

- If it is core to your differentiation: build.
- If it is commodity and well-supported: buy.
- Vector DBs: buy (Pinecone) unless cost or latency forces self-hosting.
- LLMs: usually buy (OpenAI/Anthropic) unless privacy/compliance forces local.
- Orchestration: build with thin wrappers around open-source (LangChain/LangGraph) — vendor lock-in here is expensive.
- Eval frameworks: start with open-source (RAGAS), build custom when you outgrow them.

Q9: "What's the most important non-obvious thing about deploying LLMs in production?"

Pick one and own it. Some options:

- Cost variance: same query can cost wildly different amounts depending on context length. Need cost budgets per request.
- Latency variance: LLM latency is fat-tailed. p99 matters more than p50.
- Cache layers: response cache + embedding cache + prompt cache (Anthropic) save 80% in many cases.
- Eval before/after every change: regression in faithfulness is silent and devastating.

Q10: "How does Franklin Templeton's AI strategy differ from a bank's?"

- Asset manager: focus on investment research, portfolio construction, client personalization, ESG analysis. Decisions are about allocation, not transactions.
- Bank: transactions, fraud, credit risk, customer service automation. Real-time risk dominates.

- FT's announced partnerships (Wand, Microsoft, Articul8) point at workforce augmentation, research, ops — not high-frequency decisioning.

Q11: "What is the most interesting AI paper you've read recently?"

Have one paper ready. Don't pick GPT-4 — too generic. Suggestions:

- "Lost in the Middle" (Liu et al.) — long contexts have a U-shaped attention bias. Implications for RAG chunk ordering.
- "Self-Consistency Improves Chain of Thought Reasoning" — sample multiple reasoning paths, take majority vote.
- "RAG vs Long Context: Examining Frontier LLMs" — when each wins.
- Anything from the Mistral or Llama technical reports.

Read it. Be able to discuss tradeoffs.

Q12: "If you had to cut 90% of the cost of a RAG system serving 1M queries/day, how?"

- Smaller embedding model (text-embedding-3-small is 5x cheaper than ada-002, ~95% quality)
- Smaller generator with prompt caching
- Response cache for common queries (60%+ hit rate in real systems)
- Reduce retrieved chunks from 10 to 5 after measuring impact on faithfulness
- Batch embeddings during ingestion
- Move to smaller open model (Llama 3.1 8B) for low-risk queries, escalate to GPT-4 only for hard ones (model routing)
- Negotiate enterprise rates with the model vendor

Q13: "What is causal consistency and why might it matter in a multi-agent system?"

Almost no candidate can answer this. You can, because of OnEasy. Causal consistency: if event A causes event B, every observer that sees B must also see A. In a multi-agent system where agents read and write to shared state, causal consistency prevents agents from acting on stale views of a sibling agent's output. Practical implication: every state mutation needs a logical clock or version vector if you want resumability under partial failure.

Q14: "What would you build first if you joined the team?"

This is a trick. Don't propose something specific without context. Instead:

- "I'd spend the first two weeks shadowing analysts and operations folks to understand the actual highest-pain workflow. Most production AI fails because we solve the wrong

problem. After that, I would propose a small, measurable POC against one specific workflow with clear ROI."

This answer signals Deep Srivastav's "business-first design" philosophy back at the interviewer. Powerful.

10. Resources to actually use (no fluff)

For prerequisites

- **FastAPI**: the official docs are the best tutorial.
- **LangChain**: their "Get started" guide → conceptual guide → cookbook recipes.
- **Hugging Face**: the NLP course (huggingface.co/learn). Free and excellent.
- **Transformers visualization**: Jay Alammar's "Illustrated Transformer" blog post.
- **scikit-learn**: their user guide is genuinely good.
- **Pandas**: "Modern Pandas" by Tom Augspurger.

For depth on FT-style work

- Vasundhara Chetluru's LinkedIn — read every post. Free signal on what the team thinks about.
- Deep Srivastav's public interviews (Emerj podcast, Wand AI press).
- Articul8's blog (their AWS partnership posts).
- Wand AI's blog (especially the Franklin Templeton case study).

For evaluation

- RAGAS docs.
- Anthropic's evaluation guide.
- The HHEM model card on Hugging Face.

For the agent stack

- LangGraph docs (read the whole thing, it is not long).
 - The original ReAct paper (Yao et al.) — short and influential.
-

11. Honesty pass — what to skip

You will see advice telling you to learn TensorFlow, Kubernetes, Spark, Ray, Airflow, Kubeflow, advanced calculus, GANs, CNNs, RL. For this role, in this timeframe, these are noise. Skip them.

What matters for the FT AI/ML Engineer interview, ranked:

1. RAG end to end with evaluation
2. NLP and transformer intuition
3. FastAPI + Docker + AWS deploy
4. LangChain / LangGraph
5. Classical ML fundamentals (one good project)
6. Fine-tuning awareness (one LoRA experiment)
7. Finance literacy (light)
8. Behavioral stories ready to go

Everything else is bonus.

12. Final pre-interview checklist

The day before:

- ☐ Re-skim this document
- ☐ Re-read Vasundhara's last 10 LinkedIn posts
- ☐ Run through your three core stories out loud
- ☐ Practice the "Why FT?" answer
- ☐ Make sure all your project demo URLs are live (test them!)
- ☐ Have your GitHub repos pinned and READMEs polished
- ☐ Sleep
- ☐ Coffee but not too much

In the interview:

- ☐ Slow down when answering technical questions; structure (problem → approach → tradeoff → result)
- ☐ When you don't know something, say so, then reason from first principles

- ☐ Ask 2 to 3 specific questions back: "How is the AI Platform team structured between research, engineering, and product?" or "What's the team's current biggest engineering bottleneck?"
 - ☐ End with a clear "I want this role and here's why" close
-

13. One more thing

The role description says "2 to 4 years of experience." You may technically be below that on paper. The way you compensate is by walking in able to talk about FT's exact architecture choices (LCEL, contextual compression, CAG vs RAG, hybrid retrieval, eval), pointing at live deployments of your own that demonstrate the same thinking, and having a thesis about *why* FT is doing what they're doing.

Most candidates will be technically literate. Almost none will be FT-literate. That is the gap to exploit.

Build Project 1. Get it deployed. Get one strong eval story. Read Vasundhara's posts. The rest is execution.

Good luck.